

ARCHITECTURE FOUNDATION · V0.1 · DRAFT ЗА ОБСЪЖДАНЕ

Multi-Tenant Хотелска SaaS Платформа

Channel Manager · Резервационна система · PMS — обединени около
един source of truth за наличност и цени.

Multi-tenant · Shared DB + RLS

Inventory Core = single source of truth

OTA: aggregator-first hybrid

RBAC

Demo-first delivery

Изготвено за: Konstantin / Weber BG · Статус: основа преди техническия промпт · 2026
Цел: най-стабилната, най-гъвкавата и най-добре изглеждащата система на пазара.

01 Визия и стратегия на доставка

Какво строим, защо точно така, и как стигаме най-бързо до момент, в който можем да показваме реално демо и да затваряме клиенти.

Продуктът с едно изречение

Централизирана платформа, в която **един оператор (ти) управлява много изолирани хотелски профили**; всеки хотел вижда само своите данни и работи с три модула — канален мениджър, резервации и PMS — захранвани от общо ядро за наличност и цени, със сигурна изолация на твоите бизнес данни (договори, плащания, маржове, интеграционни токени).

Трите стратегически избора (фиксирани)

Решение	Избор	Защо
Изоляция на данни	Shared DB + Row-Level Security (<code>tenant_id</code> на всеки ред, наложен на ниво база)	Индустриален стандарт. Скалира до стотици хотели, евтини миграции, силна изолация без оперативния ад на „БД на хотел“.
Първи модул	Channel Manager + Inventory Core	Това е сърцето и източникът на най-скъпите грешки (overbooking). Изградим ли го правилно, всичко друго е проекция върху него.
Доставка	Demo-first вертикален срез	Бизнес целта е да тестваме → показваме демо → затваряме клиенти. Затова правим тесен, но реален и красив срез, не цялата платформа наведнъж.

Принцип №1, който определя всичко останало

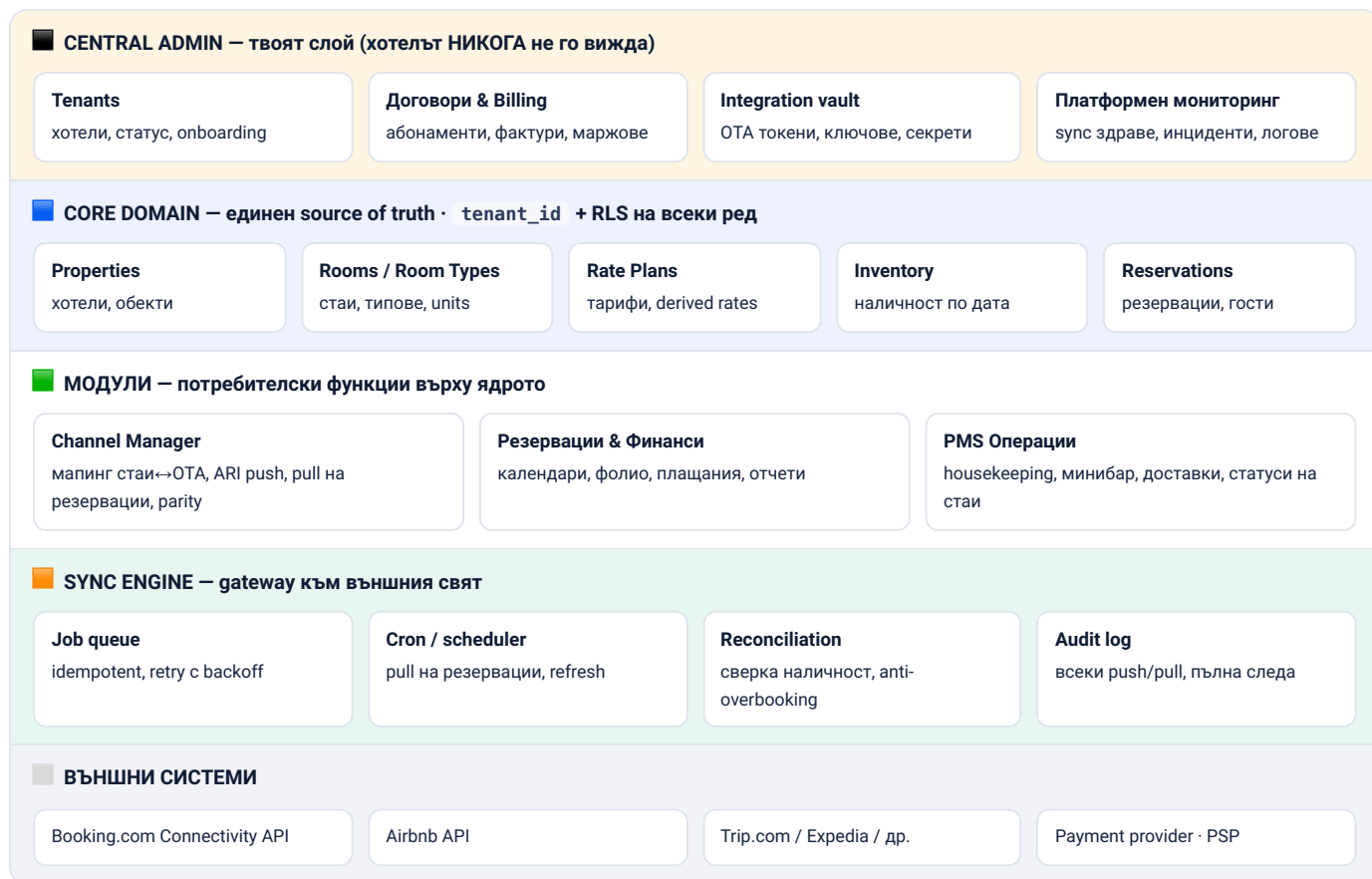
Нашата система държи истината за наличността. Booking, Airbnb, Trip.com и директните резервации са само проекции на това състояние. Всеки бгг в тази система = овербукинг = загубен клиент. Целият дизайн долу е подчинен на това.

Защо „demo-first“, а не „пълна платформа“

Пълният канал-мениджър + PMS + резервации са 12–18 месеца работа. За да затваряш клиенти не ти трябва всичко — трябва ти **един хотел, реално свързан с Booking + Airbnb, с безупречен интерфейс и нула овербукинг**. Това е постижимо за седмици, не месеци, и е достатъчно убедително за демо. Останалото се наслаждава върху същата архитектура, без преписване.

02 Архитектура на високо ниво

Слоеста архитектура. Започваме като **модулен монолит** (един deploy, ясни вътрешни граници), не микросървиси — за да сме бързи сега и да можем да разделяме по-късно без болка.



Защо модулен монолит, а не микросървиси (засега)

Микросървисите въвеждат разпределени транзакции, мрежови грешки и DevOps тежест — точно нещата, които убиват скоростта на стартиращ продукт. Модулният монолит с **ясни вътрешни граници** (всеки модул говори с ядрото през дефиниран интерфейс, не директно в чужди таблици) ти дава 90% от ползите. Когато конкретен модул (напр. Sync Engine) поиска отделно скалиране, го изваждаш — границите вече са там.

03 Multi-tenancy и изолация на данни

Моделът: Shared DB + Row-Level Security

Всяка таблица с хотелски данни носи `tenant_id` . PostgreSQL **Row-Level Security** политики налагат на ниво база, че заявка в контекста на хотел А физически не може да върне ред на хотел В — дори при бърз в приложния код. Това е разликата между „надяваме се да филтрираме правилно“ и „базата отказва да изтече данни“.

Двата изолационни периметъра

Между хотелите	Хотел ↔ твоите данни
RLS по <code>tenant_id</code>	Отделен Admin модул/схема
Хотел А не вижда хотел В	Хотелът не вижда договори, маржове, токени
На ниво база	На ниво роля + сервиз

Как се налага в практиката

- Всяка заявка минава с `SET app.tenant_id` в транзакцията.
- RLS политика: `USING (tenant_id = current_setting('app.tenant_id'))` .
- Admin операциите ползват отделна роля, която *заобикаля* RLS — но само в Admin модула.
- Интеграционните токени се пазят криптирани в отделно хранилище, не в хотелските таблици.

Уговорка за бъдеще (без преписване)

Ако дойде enterprise клиент с регулаторно изискване за физическа изолация, добавяме **schema-per-tenant** или отделна БД само за него — същият код, различен connection routing. Започваме с shared+RLS за масата; не правим отделна БД на всеки хотел (оперативно скъпо и ненужно за 99% от случаите).

Noisy-neighbor защита

Един хотел с огромен трафик не бива да задушава останалите. Затова: rate limiting per tenant на API; sync job-овете са на опашка с per-tenant квоти; тежките отчети се изпълняват асинхронно. Това го залагаме рано — добавено по-късно е болезнено.

04 Inventory Core — сърцето на системата

Тук живеят всички пари и всички овербукинги. Това е модулът, който задълбаваме първи.

Доменният модел (опростено)



- **Room Type** (напр. „Двойна стая“) с *N броя физически units* — наличността е на ниво тип, не на конкретна стая (както го правят Booking/Airbnb).
- **Rate Plan** — цена + политика (закуска, отмяна). Поддържа **derived rates** (напр. „non-refundable = base - 10%“) — задължително за конкурентност.
- **Inventory per ден** — наличен брой, цена, и **restrictions**: min/max stay, CTA (closed to arrival), CTD (closed to departure), stop-sell.

Анатомия на овербукинга — и как го убиваме

Сценарий	Защита
Две OTA продават последната стая едновременно	Атомарен decrement на наличност в транзакция с заключване на реда; никога „read-then-write“ без lock.
OTA праща резервация след като сме спрели продажба	Pull на резервации често (Booking препоръчва ~30 сек); приемаме резервацията дори при 0 наличност, после маркираме конфликт за човешка намеса — никога мълчаливо отхвърляне.
Нашата наличност и тази в OTA се разминат	Периодична reconciliation : сверяваме какво ние мислим vs какво OTA мисли, и алармираме при дрифт.
Двоен push / повторен webhook	Idempotency ключове — същото съобщение приложено два пъти дава същия резултат.

Подводен камък №1 — наличността не е просто число

Наивната реализация държи „брой свободни стаи“ и го намалява. Това гарантирано чупи при конкурентни резервации. Правилно: наличността е **деривирана** от (общо units) – (потвърдени резервации, които застъпват датата), изчислена транзакционно. По-бавно, но коректно. Никога не оптимизирай това за сметка на коректността.

05 ОТА свързаност — стратегия

Тук е най-голямата стратегическа развилка и най-подценяваният риск в плана.

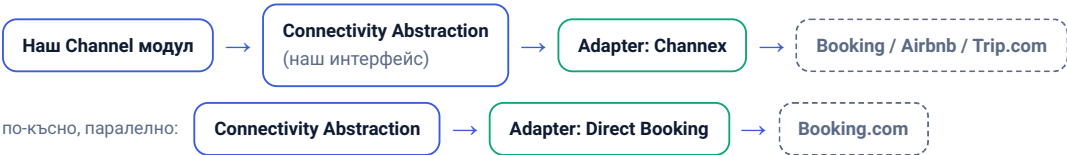
Реалността на достъпа (проверено)

Канал	Достъп	Латентност
Booking.com	Connectivity Partner програма — sandbox → продукция след сертификация. Изисква одобрение на компанията; индустрията съобщава за периоди на пауза/изчакване за нови провайдъри. Тежки изисквания: ARI 12 мес. напред, само delta updates.	pull на резервации ~всеки 30 сек
Airbnb	Invite-only. Не приемат публични заявки — те канят партньори. Privileged webhooks само за Preferred+ типа (Hostaway, Guesty, Hospitable).	webhooks (Preferred+) или polling 5–15 мин
Trip.com / Expedia	Собствени партньорски програми, подобни изисквания	варира

Подводен камък №2 — директната сертификация е чужд критичен път

Ако демото зависи от собствена сертификация с Booking/Airbnb, **датата на демо я определя чужд approval екип**, не ти. Airbnb може изобщо да не покани скоро. Това е най-вероятната причина проектът да закъсне с месеци.

Препоръка: aggregator-first hybrid зад собствена абстракция



- **Channex** е white-label connectivity-as-a-service: REST API, готов room/rate mapping iframe, вече сертифициран с 50+ канала, 99.9% SLA. Демото работи с *реални* Booking/Airbnb връзки от ден 1.
- Нашият код говори само с **нашата абстракция**. Channex е първият адаптер зад нея.
- Паралелно кандидатстваме за директни сертификации. Когато ни одобрят, добавяме `DirectBookingAdapter` зад същата абстракция и сменяме маршрута рег канал — **нула промяна за хотела, нула преписване**.

Ключово архитектурно правило

Каквото и да решим за бекенда, **никога не пишем ОТА-специфична логика директно в модулите**. Всичко минава през Connectivity Abstraction с унифициран модел (ARI update, reservation, cancellation). Така смяната на доставчик е смяна на адаптер, не хирургия в цялата система.

06 Sync Engine и роли

Sync Engine — задължителни свойства

- **Опашка с `retry + exponential backoff`** — OTA API-та падат; никога не губим update.
- **Idempotency** — повторно приложено съобщение = същото състояние.
- **Delta-only push** — пращаме само промени (Booking го изисква).
- **Reconciliation job** — периодична сверка наличност/цени.
- **Пълнен audit log** — кой, кога, какво, какъв отговор. Това е и debug инструмент, и доверие пред клиента.
- **Dead-letter queue** — неуспели след N опита отиват за човешки преглед, не изчезват.

Подводен камък №3 — тихите грешки

Най-лошият channel manager е този, който се проваля мълчаливо (XML polling на 30 мин, без логове). Хотелът разбира за проблема от ядосан гост. Нашето конкурентно предимство: **видим sync статус и алармиране** — хотелът винаги знае дали е синхронизиран.

RBAC — ролево-базиран достъп (не хардкоднати роли)

Роля	Вижда	Може	НЕ вижда
Super-admin (ти)	Всички хотели, договори, billing, токени	Всичко, платформена администрация	—
Hotel owner	Само своя хотел, всички модули	Управлява персонал, вижда финанси на хотела	Твоите маржове, договори, други хотели
Manager	Резервации, цени, отчети, PMS	Променя цени/наличност, отчети	Billing към теб, токени
Receptionist	Календар, резервации, гости	Чек-ин/аут, нова резервация	Финансови отчети, цени конфигурация
Housekeeping	Статуси на стаи, задачи	Маркира стая чиста/мръсна, минибар	Резервации, финанси, гости детайли

Реализация: **permissions като scopes** (напр. `reservations:write` , `rates:write` , `billing:read`), ролите са просто набори от scopes. Така добавяш нова роля или custom права без да пипаш код.

07 Технологичен стек и сигурност

Препоръчан стек (за обсъждане)

Слой	Препоръка	Защо
База данни	PostgreSQL	Native RLS (нашата изолация), транзакции, надеждност, JSON за гъвкавост.
Backend	Node/TypeScript (NestJS) или Python (FastAPI/Django)	Силен типов модел за домейна; голям talent pool. Изборът зависи от твоя екип.
Job queue	Redis + BullMQ / Celery	Sync Engine – retry, scheduling, dead-letter.
Frontend	React + TypeScript, дизайн система	Уникалният интерфейс е конкурентно предимство – заслужава първокласен фронтенд.
Connectivity	Channex (адаптер #1)	Реални OTA връзки без чакане на сертификация.
Hosting	Управляван Postgres + контейнери	По-малко DevOps, повече продукт.

Сигурност — базови изисквания

- **RLS** като последна линия на изолация между хотели.
- **Криптирани интеграционни токени** (vault / KMS), никога в plain text, никога видими за хотела.
- **OAuth-стил токени** за API достъп; кратко живеещи access + refresh.
- **Audit log** на чувствителни действия (промяна на цени, достъп до гости).
- **GDPR** — гостите са лични данни (имена, имейл, понякога документи). Нужни: retention политики, право на изтриване, data processing agreement.
- **Плащания** → **не пипаме карти директно**. Tokenization през PSP (Stripe и под.); PCI обхватът ни остава минимален.
- **Rate limiting** per tenant (noisy-neighbor).
- **Backups + point-in-time recovery** от ден 1.

Подводен камък №4 — GDPR и плащания не са „по-късно“

Хотелските данни включват лични данни на гости и (понякога) картови данни. Ако ги дизайннем грешно от старта, преработката е скъпа и рискова. Решение от ден 1: гостите са отделен, контролиран домейн с retention; картови данни никога не докосват нашата база — само токени от PSP.

08 Какво ме притеснява — и какво ще ни направи №1

Подводните камъни, обобщено

Риск	Влияние	Митигиране
Овербукинг при конкурентни резервации	Загубени клиенти, репутация	Транзакционна, деривирана наличност + reconciliation
Зависимост от чужда OTA сертификация	Демото закъснява с месеци	Aggregator-first зад собствена абстракция
Тихи sync грешки	Загуба на доверие	Видим статус, алармиране, dead-letter queue
GDPR / картови данни събрани рано	Скъпа преработка, правен риск	Гости-домейн + PSP tokenization от старта
Преждевременни микросървиси	Бавно, скъпо, крехко	Модулен монолит с ясни граници
Vendor lock-in в Channex	Зависимост, маржове	Абстракция → лесна смяна/добавяне на директни адаптери
Mapping грешки (стая↔OTA)	Грешни цени/наличност	Контролиран mapping UI + валидации + dry-run преди go-live
Скоуп пълзене (всичко наведнъж)	Никога не стигаме до демо	Demo-first вертикален срез

Какво ще ни направи най-добрите (не просто „поредния channel manager“)

Доверие чрез прозрачност

Видим sync статус в реално време, ясни логове, „последно синхронизирано преди X“ на всеки канал. Конкурентите крият това.

Нула-овербукинг гаранция

Архитектурно гарантирана коректност на наличността — продаваме спокойствие, не просто софтуер.

Уникален интерфейс

Един календар, в който наличност, цени и резервации от всички канали се виждат и редактират на едно място, с моментален визуален feedback. Това е продуктовото предимство.

Гъвкавост за бъдеще

Адаптерна connectivity + RBAC scopes + модулни граници = добавяме канали, роли и модули без преписване.
„Непробиваемо и гъвкаво“ = тези три неща.

09 Пътна карта до демо и отворени въпроси

Фазиран план (demo-first)

Фаза	Какво	Резултат
0 · Основа	Postgres + RLS scaffold, auth, RBAC scopes, tenant модел	Сигурен multi-tenant скелет
1 · Inventory Core	Room types, rate plans, derived rates, транзакционна наличност, restrictions	Source of truth работи и е коректен
2 · Channel (1 хотел)	Connectivity Abstraction + Channex адаптер, mapping, ARI push, pull на резервации	Реални Booking+Airbnb връзки
3 · UI за демо	Уникалният унифициран календар + dashboard, sync статус	Красиво, убедително демо → клоузваме
4 · Резервации/ Финанси	Фолио, плащания (PSP), отчети	Пълен резервационен цикъл
5 · PMS	Housekeeping, минибар, доставки	Операции в хотела
6 · Скалиране	Директни OTA адаптери, повече хотели, enterprise изолация при нужда	Растеж без преписване

Въпроси, на които да отговорим преди техническия промпт

1. **Тип имоти** — класически хотели (room types с много units) или и апартаменти/къщи (Airbnb стил, 1 unit)? Това променя модела на наличност.
2. **Стек** — какъв е твоят/екипният опит: Node/TS или Python? Това определя backend избора.
3. **Плащания** — приемаме ли плащания в платформата (PSP, фолио) или само следим? Влияе на PCI/финансовия модул.
4. **Channex** — окей ли е като пръв connectivity бекенд (потвърждение), и ползваме ли техния mapping iframe или собствен UI?
5. **Мащаб на демото** — 1 хотел + Booking + Airbnb достатъчно ли е за първото демо, или трябва и Trip.com?
6. **Multi-property вериги** — един „хотел owner“ управлява ли няколко обекта, или 1:1? Влияе на tenant модела.
7. **Език/локализация** — само BG, или и EN/мулти-валута от старта (за международни хотели)?

Следваща стъпка

Като пуснеш 3-те документа, ще обединя тяхната конкретика с тази основа и ще ти дам **топ технически промпт** + ще довършим тези въпроси, преди да започнем изграждането. Този PDF е скелетът, върху който всичко се наслагва.